

Adam

Adaptive Moment Estimation

Vad gör en optimiser?

Optimiser är den algoritm som bestämmer hur vikterna i en ML modell justeras.

Den enklaste optimisern är **Stochastic Gradient Descent** (SGD).

Vad gör en optimiser?

Gradient Descent: räknar ut lutningen av loss funktionen och tar steg i motsatt riktning för att minimera felet.

Stochastic = stokastisk, slumpmässig

SGD: Börjar med slumpmässiga vikter. Beräknar lutningen av lossfunktionen för en batch. Justerar vikterna. Upprepa.

Eftersom varje batch är slumpmässig så fluktuerar SGD i sin väg mot minimum

Momentum: Medan SGD hoppar lite hit och dit på vägen mot minimum (eftersom varje batch är slumpmässig) så ger Momentum just momentum till optimeringen genom att lägga till ett "minne" av tidigare steg.

Optimiser evolution

Först hade vi SGD (Stochastic Gradient Descent) som är vår enklaste optimiser
Vilken sen blev mer effektiv med Momentum

2014 uppfanns Adam och presenterades 2015 i en forskningsartikel

2018 förbättrades Adam av en annan forskargrupp, vilket ledde till AMSGrad

Finns många fler optimisers förutom SGD, Momentum, Adam och AMSGrad, tex NAG, RMSProp, Lion, LAMB, etc.

Adam 2015

Published as a conference paper at ICLR 2015

ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION

Diederik P. Kingma*

University of Amsterdam, OpenAI

dpkingma@openai.com

Jimmy Lei Ba*

University of Toronto

jimmy@psi.utoronto.ca

ABSTRACT

We introduce *Adam*, an algorithm for first-order gradient-based optimization of stochastic objective functions, based on adaptive estimates of lower-order moments. The method is straightforward to implement, is computationally efficient, has little memory requirements, is invariant to diagonal rescaling of the gradients, and is well suited for problems that are large in terms of data and/or parameters. The method is also appropriate for non-stationary objectives and problems with very noisy and/or sparse gradients. The hyper-parameters have intuitive interpretations and typically require little tuning. Some connections to related algorithms, on which *Adam* was inspired, are discussed. We also analyze the theoretical convergence properties of the algorithm and provide a regret bound on the convergence rate that is comparable to the best known results under the online convex optimization framework. Empirical results demonstrate that Adam works well in practice and compares favorably to other stochastic optimization methods. Finally, we discuss *AdaMax*, a variant of *Adam* based on the infinity norm.

Adam 2018 => AMSGrad

Published as a conference paper at ICLR 2018

ON THE CONVERGENCE OF ADAM AND BEYOND

Sashank J. Reddi, Satyen Kale & Sanjiv Kumar

Google New York

New York, NY 10011, USA

{sashank, satyengkale, sanjivk}@google.com

ABSTRACT

Several recently proposed stochastic optimization methods that have been successfully used in training deep networks such as RMSPROP, ADAM, ADADELTA, NADAM are based on using gradient updates scaled by square roots of exponential moving averages of squared past gradients. In many applications, e.g. learning with large output spaces, it has been empirically observed that these algorithms fail to converge to an optimal solution (or a critical point in nonconvex settings). We show that one cause for such failures is the exponential moving average used in the algorithms. We provide an explicit example of a simple convex optimization setting where ADAM does not converge to the optimal solution, and describe the precise problems with the previous analysis of ADAM algorithm. Our analysis suggests that the convergence issues can be fixed by endowing such algorithms with “long-term memory” of past gradients, and propose new variants of

Vad är Adam?



Momentum

Minns vilken riktning vi rörde oss nyligen – håller farten uppe



Adaptiv learning rate

Varje parameter får sin egna effektiva learning rate

Adam = SGD + Momentum + Adaptiva learning rates
Kombinationen gör optimering mer effektiv och stabil

Adams två moment

m

Första momentet – Riktning

Rörligt medelvärde av gradienten

- Om medelvärdet är högt pekar gradienterna i samma riktning – full fart framåt
- Fungerar som momentum

v

Andra momentet – Skalning

Rörligt medelvärde av gradientens kvadrat

- Stora gradienter → mindre steg
- Små gradienter → större steg

Varje parameter lär sig i sin egen takt

Tillsammans ger m och v både riktning och rätt steglängd per parameter

Video

<https://www.youtube.com/watch?v=MD2fYip6QsQ>

04:34, 06:34 och 15:33 för visualiseringar av olika optimisers

Och för mer om formulan se:

<https://www.geeksforgeeks.org/deep-learning/adam-optimizer/>

Konkret exempel för vårt case

Tänk dig att du tränar ett nätverk på bilder av kläder. Vissa pixlar i hörnen är nästan alltid svarta. De förändras sällan och ger små gradienter. Andra pixlar mitt i bilden varierar mycket och ger stora gradienter.

Med SGD+momentum får alla pixlar samma stegstorlek.

Med Adam lär sig parametrar kopplade till de lugna hörn-pixlarna att ta större steg, medan parametrar med stora gradienter automatiskt tar mindre steg för att inte hoppa förbi målet.

Minnesanvändning (GPU) - Adam kräver mer minne

SGD utan momentum



Sparar bara gradienten

SGD med momentum



Gradient + m (rörligt medelvärde)

Adam



Gradient + m + v

Trots 3x minneskrav är Adam ofta värt det – snabbare konvergens = färre epoker = lägre total energiförbrukning för stora modeller

Adam är effektivare än SGD



Snabbare konvergens

Modellen når bra accuracy på färre epoker eftersom varje steg är mer välriktat



Stabilare träning

Adaptiv skalning minskar risken att hoppa förbi minimum – loss-kurvan är jämnare



Lägre energiförbrukning

Färre träningssteg = lägre energiförbrukning för stora modeller

Ideér för vår uppgift



Ideér för vår uppgift

